Patient-Aware Agentic Cyber-Resilience for the Internet of Medical Things: Guarded LLM Incident Response with Blockchain Provenance

Rohith V[a], Prathiv R M[a], Manimaran S[a]

[a]*Amrita School of Artificial Intelligence, Coimbatore, Amrita Vishwa Vidyapeetham, India*

Abstract

Hospitals depend on networked medical devices, yet an automated cyber response can itself harm a patient: isolating an attacked ventilator stops breathing support. Existing Internet of Medical Things (IoMT) detectors end at classification, and language-model incident-response agents give no guarantee against wrong or manipulated output. We propose a four-phase, patient-aware cyber-resilience framework combining an XGBoost detector with calibrated confidence, five language-model agents treated as untrusted, deterministic grounding guards, rule-based risk and policy engines that require a human for severe actions on patient-dependent devices, and a Hyperledger Fabric ledger for tamper-evident provenance. On the full CICIoMT2024 test split (1,614,182 flows) the detector reaches 99.52% accuracy. Across about 262,000 high-stakes attacks the full system executed no unsafe autonomous action, against 99.72% for a fixed-response baseline, and an exhaustive check of 3,168 policy inputs found no violation. Injected agent faults caused two unsafe actions without the guards and none with them; the ledger detected every tested tampering.

Keywords: Internet of Medical Things, cyber-resilience, agentic AI, large language models, patient safety, Hyperledger Fabric, CICIoMT2024

1. Introduction

The Internet of Medical Things (IoMT) connects infusion pumps, patient monitors, ventilators and imaging equipment to hospital networks, and with that connectivity comes a large attack surface. Research on IoMT security has produced a large body of intrusion-detection work, which recent reviews

organise by method, dataset and evaluation metric [1]. The CICIoMT2024 benchmark [2], collected from a testbed of IoMT devices under attack over several protocols, has become a common test bed. Detectors built on it now report high accuracy: an enhanced long short-term memory network called L2D2 reaches 98% over 19 classes [3], a reinforcement-learning hybrid reaches 0.9958 binary accuracy [4], and an interpretable XGBoost fusion reaches 0.97 [5].

The objective of this work is to move from *detecting* an attack on a medical device to *responding* to it safely. A response is safe only if it does not harm the patient who depends on the device, and it is useful only if it can be explained and audited afterwards.

In a clinical network the most secure response is not always the safest one. A fixed playbook that isolates any device under attack will, on a ventilator, remove breathing support to stop a network flood. Language-model agents have recently been proposed to automate incident response [6, 7], but a model can be wrong, can be misled by text inside the traffic it reads, and can claim that a device has recovered while the attacker is still active. Blockchain has been proposed to protect health records and evidence [8, 9], yet it is rarely measured on a real ledger together with detection and response. The open problem is therefore: given IoMT traffic and a registry of device criticality, choose an automated response whose risk to a patient-dependent device is controlled *by construction*, whatever an AI component says, and keep a tamper-evident record of every step.

Closely related work from our own institution combines smart contracts with encryption for medical data sharing [10] and builds a smart-contract blockchain for a healthcare IoT network [11], while cyber-physical work applies machine learning to attack classification and anomaly detection in power systems [12, 13]. These share our concern with trustworthy automation in safety-critical infrastructure, but none of them gates an automated response on the clinical state of a device.

This paper makes the following contributions.

1. A patient-aware response architecture that separates reasoning (five language-model agents) from decision (deterministic Risk and Policy engines), so that no agent can by itself cause a disruptive action on a high-stakes device.
2. Three deterministic grounding guards that treat agent output as untrusted: the device registry overrides a model's clinical-context claim, a

recovery claim is overridden if attack traffic continues, and an answer about the wrong incident is rejected.

3. A calibrated confidence gate whose threshold is chosen on held-out validation data and scored once on test, which reduces unnecessary autonomous isolation on false alarms.
4. Blockchain provenance measured on a real network: incident, decision, approval, response and recovery are written to a Hyperledger Fabric ledger, and integrity, tamper evidence, latency and concurrency are measured on a real two-organisation deployment.
5. A safety-focused evaluation with real experiments at several levels: all 1,614,182 official CICIoMT2024 test flows, five training seeds of six classifiers, bootstrap confidence intervals, an exhaustive enumeration of the decision rules, a sensitivity study of the clinical device mix, and a fault-injection study of the agentic layer with the guards on and off.

The rest of the paper is organised as follows. Section 2 reviews sixteen related works and states the identified gaps. Section 3 presents the proposed framework, its four phases and algorithms. Section 4 gives the experimental setup, dataset and results. Section 5 compares the framework with existing work. Section 6 concludes with directions for future research.

2. Literature review

Table 1 summarises the sixteen works cited in this paper, grouped by topic: IoMT intrusion detection and datasets, blockchain for healthcare and forensic evidence, language-model agents for incident response, and machine learning for cyber-physical power systems. For each we state the algorithm, its advantage, and its drawback *with respect to our objective of safe, auditable automated response*. The drawbacks are therefore not criticisms of the works' own aims.

Table 1: Sixteen related works (2021–2026): algorithm, advantage and drawback relative to this work.

Ref.	Algorithm / approach	Advantage	Drawback
IoMT intrusion detection and datasets			
[1]	Systematic literature review of intrusion-detection systems for the IoMT.	Organises methods, datasets and metrics; identifies open challenges.	A survey: no system, and no link from detection to a safe response.
[2]	CICIoMT2024: multi-protocol attack dataset from 40 IoMT devices (WiFi, MQTT, Bluetooth).	Real device traffic; 18 attack types; official capture-level split.	Network flows only; no clinical information about the devices.
[3]	Enhanced LSTM (L2D2) on CICIoMT2024, 19 classes.	98% accuracy on the real multi-class benchmark.	Detection only; no response, clinical context or provenance.
[4]	CNN-LSTM features with DQN and PPO decision learners; mutual-information feature selection.	0.9958 binary accuracy; learned decision policy.	18-class accuracy only 0.7773; decisions concern classification, not device impact.
[5]	Tuned XGBoost with logistic-regression late fusion and SHAP; cost-aware.	Interpretable; fusion cut false negatives from 361 to 213.	Generic security cost; no patient impact; no automated response.
[14]	Stacking, bagging and boosting of random forest and SVM on WUSTL-EHMS-2020.	98.88% accuracy with stacking; uses patient-biometric features.	One dataset, as the authors note; detection only.

Ref.	Algorithm / approach	Advantage	Drawback
[15]	Soft-voting XGBoost and two SVMs with PCA features; validated on TON-IoT and CICIDS 2017.	98.04% accuracy; cross-dataset validation.	Detection only; no response or clinical gating.
Blockchain for healthcare and evidence			
[8]	Systematic review of Hyperledger Fabric for health-record security (57 studies).	Collects the privacy, integrity and traceability benefits of Fabric.	A review: no implementation and no incident response.
[9]	Prototype Hyperledger Fabric security architecture for IoMT health monitoring.	A real Fabric prototype aimed at IoMT.	Protects monitoring data, not an automated response decision.
[16]	Blockchain digital-forensics scheme for industrial IoT: smart contracts, token access control, batch consensus.	Tamper-proof, non-repudiable evidence chain; faster than DPoS in simulation.	Industrial setting, evaluated in simulation; no clinical safety notion.
[10]	Smart contracts with an encryption scheme for privacy-aware medical data sharing.	Privacy-preserving sharing of medical data on a blockchain.	Data sharing, not incident response; no detector or device gating.
[11]	Smart-contract-based blockchain for a healthcare IoT network.	Applies smart contracts directly to healthcare IoT.	Concerns data handling in the network; no attack detection or response policy.
Language-model agents for incident response			
[6]	IRCopilot: four cooperating LLM sessions that mirror incident-response phases (preprint).	Role split reduces context loss and hallucination.	No deterministic guard on model output; not medical; no clinical gating.

Ref.	Algorithm / approach	Advantage	Drawback
[7]	Fine-tuned 14B LLM with perception, reasoning, planning and action, plus in-context adaptation (preprint).	Recovery up to 23% faster than frontier models.	Optimises recovery time; patient harm not modelled; a five-action plan takes about 20 minutes on an A100.
Machine learning for cyber-physical power systems			
[12]	Man-in-the-middle attack modelling in a smart grid with machine-learning attack classification.	Shows ML classification of attacks on a physical process.	Power systems, not medical; classification without a safe response layer.
[13]	AI-driven cybersecurity framework for anomaly detection in power systems.	Anomaly detection for a safety-critical physical infrastructure.	Detection only; no response gating on the physical consequence.

Two observations follow from this review. First, the blockchain studies [8–11, 16] protect data, access or evidence, but none reports a ledger written by a detect-and-respond pipeline and measured for tamper detection and latency. Second, the cyber-physical studies [12, 13] show that machine-learning detection is also central in power systems, where, as in a hospital, the physical process can be harmed by a naive automatic response. The detection papers [1, 3–5, 14, 15] end at classification; the response papers [6, 7] are not medical and are not guarded against wrong model output.

The review leaves three gaps, which together form the problem statement of this paper. The first is that IoMT detectors end at classification and do not decide what to do to a device a patient depends on. The second is that language-model incident-response agents have no hard guarantee that a wrong, manipulated or malformed answer cannot trigger a harmful action. The third is that provenance for incident handling is described but seldom implemented and measured end to end on a real ledger. This work closes the three gaps together: it adds a patient-aware, deterministic decision layer after a real detector, treats the agents as untrusted, and records the whole

incident on a real Fabric network.

3. Proposed work

3.1. Design principles

Three principles shape the framework. First, agents propose and code decides: language-model agents investigate and plan, but only deterministic code (the Risk and Policy engines) decides whether an action may run without a human. Second, the registry is the ground truth: the clinical criticality of a device and whether a patient depends on it come from a device registry, never from a model's description. Third, everything is recorded off the critical path: every step is written to a ledger through an outbox, so a slow ledger cannot delay containment.

3.2. Architecture

Figure 1 shows the architecture. Network traffic is classified by the detector (Phase I). The agents turn an alert into an investigation, a threat assessment, a clinical-context statement and a proposed response (Phase II). The Risk Engine scores the situation and the Policy Engine routes the action to autonomous execution or to a human (Phase III). After execution a recovery check closes or reopens the incident (Phase IV). The device registry feeds the guards and the engines (dashed arrows), and a Hyperledger Fabric ledger records the incident, decision, approval, response and recovery. The only components that contain a language model are the agents of Phases II and IV; everything else in the figure is deterministic code.

3.3. Phases of the framework

The framework runs in four phases (Figure 2). Phase I detects an attack and attaches a calibrated confidence. Phase II lets the agents investigate and propose one action, with every answer checked by the guards. Phase III scores the situation and decides, through the Policy Engine, whether the action runs alone or waits for a human. Phase IV verifies the outcome and closes or reopens the incident. The ledger bar at the bottom of Figure 2 shows that the ledger is written at the end of Phases I, III and IV, and never sits on the path between a decision and its execution.

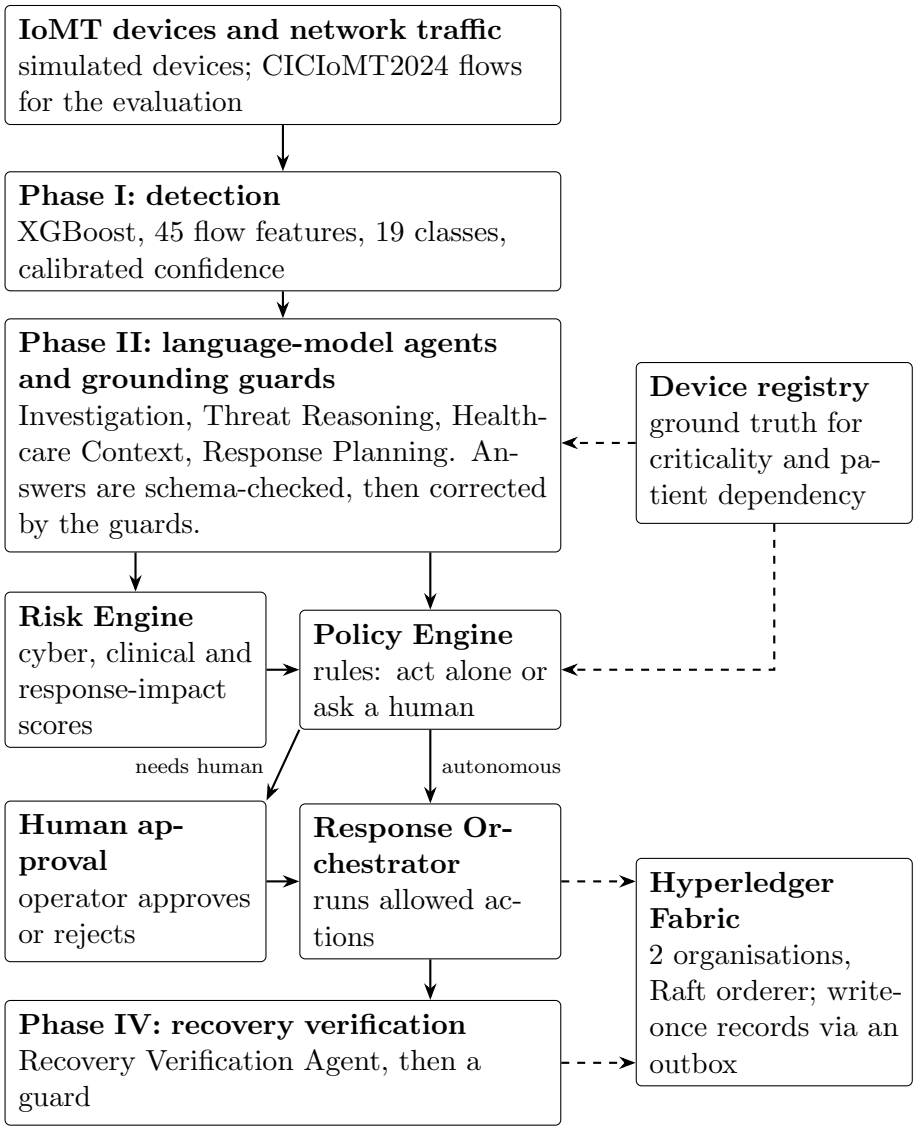


Figure 1: Architecture of the proposed framework. Solid arrows: data and control flow. Dashed arrows: registry reads and ledger writes.

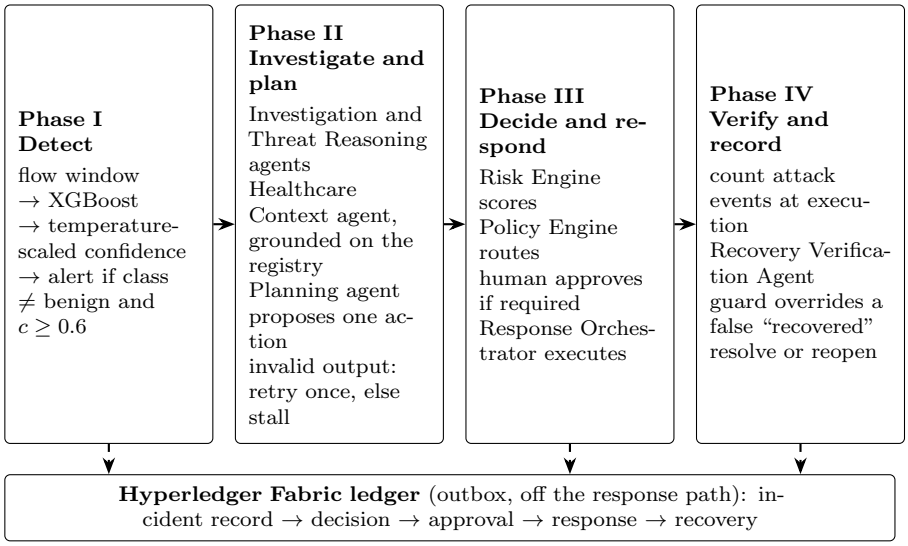


Figure 2: The four phases of the framework and the points at which ledger records are written (dashed arrows).

3.4. Phase I: detection and calibrated confidence

A flow window with feature vector $\mathbf{x}$ is classified by an XGBoost model into one of $K = 19$ classes (Benign plus 18 attack types). With logits $z_k(\mathbf{x})$, the calibrated probability and confidence are

$$p_k(\mathbf{x}) = \frac{\exp\big(z_k(\mathbf{x})/T\big)}{\sum_{j=1}^{K} \exp\big(z_j(\mathbf{x})/T\big)}, \qquad c(\mathbf{x}) = \max_k p_k(\mathbf{x}), \tag{1}$$

where the temperature T is fitted by minimising the negative log-likelihood on a validation split ($T = 0.917$ in our run). An alert is raised when the predicted class $\hat{y}$ is not Benign and $c \geq \theta$ with $\theta = 0.6$. An optional *confidence gate* additionally forbids autonomous action when $c < \tau$; τ is chosen on validation data only (Section 4.6). Equation (1) is the only place the temperature enters; it rescales confidence without changing which class has the largest probability.

3.5. Phase II: guarded agents

Five agents are used: Investigation, Threat Reasoning, Healthcare Context, Response Planning and Recovery Verification (the last in Phase IV). Each returns JSON that must satisfy a schema. A parse or schema failure triggers one retry; a second failure stalls the incident in a safe state (for example

`investigation_stalled`) instead of guessing. Three deterministic guards then correct what the agents claim: the clinical-context guard, the recovery-claim guard and the incident-identity guard.

$$\kappa' = \kappa_{\text{reg}}(d), \qquad \pi' = \pi_{\text{reg}}(d), \tag{2}$$

$$i' = \max(i, \text{HIGH}) \;\text{ if } \kappa_{\text{reg}}(d) = \text{HIGH} \wedge \pi_{\text{reg}}(d), \tag{3}$$

$$\text{success}' = \text{success} \wedge (n_{\text{after}} \leq n_{\text{exec}}), \tag{4}$$

$$\text{accept } o \iff \text{id}(o) = \text{id}(\text{incident}). \tag{5}$$

Equations (2) and (3) define the clinical-context guard, Eq. (4) the recovery-claim guard and Eq. (5) the incident-identity guard. In these equations κ is device criticality, π is patient dependency, i is the stated clinical impact, d is the device, o is an agent's output, n_{exec} is the count of attack events observed when the action was executed and n_{after} the count at verification. The recovery-claim guard uses an event count, not a clock, so it needs no synchronised time. Every correction is logged and stored in the incident trace next to the agent's original output.

3.6. Phase III: Risk Engine and Policy Engine

The Risk Engine is an explicit rule table, not a learned model, so each score can be audited. With threat severity s and the Threat Reasoning Agent's confidence $\gamma \in [0, 1]$, device criticality κ, clinical impact i and patient dependency $\pi \in \{0, 1\}$:

$$R_{\text{cyber}} = b_s \, (0.5 + 0.5\,\gamma), \tag{6}$$

$$R_{\text{clin}} = \min\{1,\ \max(b_\kappa + \Delta_i,\ 0.55\,\pi)\}. \tag{7}$$

The base scores are $b_s \in \{0.15, 0.40, 0.70, 0.90\}$ for low, medium, high and critical severity, $b_\kappa \in \{0.10, 0.35, 0.60\}$ for low, medium and high criticality, and $\Delta_i \in \{0, 0.05, 0.15, 0.25\}$ for low to critical clinical impact. The 0.5 floor in (6) stops a low-confidence severe assessment from scoring near zero, and the 0.55 floor in (7) means a device a patient depends on never scores low clinical risk. The third score, *response impact*, says how harmful the *action itself* would be on this device. Table 2 gives its values: the same severe action is rated medium impact on a device without a patient, high on a patient-dependent device and critical on a high-criticality, patient-dependent device.

Table 2: Response impact of an action, by its disruption tier and the device context.

Action tier (examples)	HIGH criticality and patient-dependent	Patient-dependent	Not patient-dependent
Severe: shutdown, isolate critical, quarantine segment, failover	Critical	High	Medium
Moderate: isolate non-critical, restrict communication, restart service, block traffic	High	Medium	Low
Non-disruptive: rotate credentials, revoke session, activate backup	Medium	Low	Low

The Policy Engine routes the single action the planner recommends, and the first matching rule wins. A patient-dependent device with a severe action of shutdown or critical isolation always goes to a human. Otherwise an action whose response impact is low or less runs autonomously; a medium impact runs autonomously if the device is not patient-dependent and goes to a human if it is (a conservative gap-filling rule); and a high or critical impact always goes to a human. No rule reads a model's opinion of the device or the attack. The optional confidence gate of Section 4.6 is checked last and can only turn an autonomous decision into a human one, never the reverse. Each decision carries a stable rule identifier so a reviewer can see which rule fired, and Section 4.9 checks these rules over their entire input space.

3.7. Phase IV: recovery verification and provenance

After an action runs, the system records n_{exec}, the Recovery Verification Agent judges whether the device is safe, and the recovery-claim guard overrides a success claim if attack events have continued. A failed check reopens the incident. Five write-once records go to Hyperledger Fabric 2.5.9: the incident, decision, approval, response and recovery records. Each stores an evidence hash rather than the evidence. A transaction commits only if a majority of the two organisations (Hospital and SecurityOps) endorse it, and the client waits for both peers' commit events before reporting success. A verification function recomputes a hash and checks that it is on the ledger.

3.8. Algorithms

Algorithm 1 is the end-to-end procedure for one flow window. Algorithm 2 is the guarded agent call used inside it. The detector runs on every flow window, but only an alert opens an incident, so the agents, engines and ledger run at incident rate, not flow rate. Because the engines and guards are plain code, they can be unit-tested exhaustively; the language-model calls are the only non-deterministic step, and a failure there stalls safely or is corrected, never executed. The implementation is a Python backend with FastAPI, XGBoost for the detector, a Node.js chaincode for Fabric, and a simulated hospital of devices for the response and recovery steps.

Algorithm 1 shows where each earlier component sits: the detector (line 1) uses Eq. (1), the clinical-context guard (line 8) corrects the Healthcare Context Agent before anything is scored, and the Risk and Policy engines (lines 11–12) see only corrected inputs. The ledger calls are plain writes between steps. In Algorithm 2 the only way out of the loop other than a valid answer is a safe stall, which is why a model that returns garbage, or an answer about the wrong incident, cannot cause an action.

4. Results

4.1. Experimental setup

The evaluation has eight parts, each answering one question about one part of the framework. The detector is measured on real data (Sections 4.4 and 4.5); its confidence and the confidence gate are checked (Section 4.6); the patient-safety behaviour of the decision layer is measured against baselines and ablations (Section 4.7) and stress-tested over different clinical device mixes (Section 4.8); the decision rules are verified over their whole input space (Section 4.9); the agentic layer is attacked with injected faults (Section 4.10); and the blockchain is measured on a real Fabric network (Section 4.11). Every figure and table below is produced by a script in the project repository and written to a report file; none is typed by hand. The repository is available on request from the authors (TODO: add the public repository link).

The backend is Python (FastAPI) with an XGBoost detector (150 trees, depth 8, learning rate 0.2, histogram method, 256 bins) and a random-forest comparator (100 trees). Fabric 2.5.9 runs natively on one host with 2 CPUs: two organisations (Hospital, SecurityOps) with one peer each, one Raft orderer with a 1 s batch timeout, a Node.js chaincode through an external builder, and a majority endorsement policy. Detector training uses fixed seeds (seed 42

Algorithm 1 Incident handling in the proposed framework

Require: flow window $\mathbf{x}$ on device d; registry $\mathcal{R}$; thresholds θ, optional τ

$(\hat{y}, c) \leftarrow \textsc{Detect}(\mathbf{x})$ ▷ Eq. (1)
if $\hat{y} =$ Benign **or** $c < \theta$ **then**
    return
end if
open incident u and call $\textsc{Ledger.recordIncident}(u)$
$E \leftarrow \textsc{GuardedCall}(\text{Investigation}, u)$
$A \leftarrow \textsc{GuardedCall}(\text{ThreatReasoning}, E)$
$C \leftarrow \textsc{GuardedCall}(\text{HealthcareContext}, u)$
$C \leftarrow \textsc{ContextGuard}(C, \mathcal{R}[d])$ ▷ Eqs. (2), (3)
$P \leftarrow \textsc{GuardedCall}(\text{ResponsePlanning}, A, C)$
$a \leftarrow P.\text{recommended}$
$S \leftarrow \textsc{RiskEngine}(A, C, P)$ ▷ Eqs. (6), (7); Table 2
$(h, \rho) \leftarrow \textsc{PolicyEngine}(S, C, a, c, \tau)$ ▷ rules of Section 3.6
$\textsc{Ledger.recordDecision}(u, a, \rho)$
if h **then**
    wait for the operator; $\textsc{Ledger.recordApproval}(u)$
    if rejected **then**
        return
    end if
end if
$n_{\text{exec}} \leftarrow \textsc{CountAttackEvents}(d)$
execute a and call $\textsc{Ledger.recordResponse}(u)$
$V \leftarrow \textsc{GuardedCall}(\text{RecoveryVerification}, u)$
$V \leftarrow \textsc{RecoveryGuard}(V, n_{\text{exec}})$ ▷ Eq. (4)
$\textsc{Ledger.recordRecovery}(u)$
mark u resolved if $V.\text{success}$, otherwise reopened

Algorithm 2 GuardedCall: schema validation, retry and identity check

Require: agent g, prompt q, schema Σ, incident id u

```
for attempt = 1, 2 do
    o ← LLM(g, q)
    if o parses, satisfies Σ and id(o) = u then ▷ incident-identity guard, Eq. (5)
        return o
    end if
end for
stall the incident safely (no action is taken)
```

for the main model; seeds 0–4 for the multi-seed study) and the device-context sampling of the safety study uses five fixed seeds.

4.2. Dataset

We use the WiFi and MQTT release of CICIoMT2024 [2], a benchmark of network flows from a testbed of IoMT devices under attack. MQTT (Message Queuing Telemetry Transport) is a publish-subscribe protocol widely used by medical sensors. The attacks fall into six families: MQTT attacks (floods and malformed data), reconnaissance scans, ARP spoofing, and TCP/IP denial-of-service (DoS) and distributed denial-of-service (DDoS) floods over ICMP, SYN, TCP and UDP. The official capture-level split is kept: no random re-splitting and no tuning on the test split. Table 3 gives the dimensions. The test split is dominated by volumetric floods (about 1.5 million of its 1.6 million flows), so overall accuracy is high by construction and macro-F1, which weights all 19 classes equally, is the more informative figure.

Table 3 also shows the main limit of the data for our purpose: the files describe network flows only, so which device was attacked, how critical it is and whether a patient depends on it must be supplied separately. Sections 4.7 and 4.8 do this by sampling, and say so wherever it matters.

4.3. Evaluation metrics

For the detector we report accuracy, macro-F1 (the unweighted mean of per-class F1 over 19 classes, or over six attack families), the attack-detection rate (the share of attack flows flagged as any attack), the benign false-positive rate (the share of benign flows flagged as an attack), the area under the ROC curve (AUC) and average precision for attack versus benign, the

Table 3: CICIoMT2024 (WiFi and MQTT) as used in this work.

Property	Value
Official training rows	7,017,530 (training capped at 300,000 per class for the main XGBoost model, 30,000 for the main random forest)
Rows used by the main models	2,975,443 (XGBoost); 441,318 (random forest)
Official test rows (all used, none sampled)	1,614,182: 37,607 benign, 1,576,575 attack
Features per flow	45 (the dataset's own flow features)
Classes	19: Benign and 18 attacks (MQTT floods and malformed data; TCP/IP DDoS and DoS; reconnaissance; ARP spoofing)
Clinical device context	Not in the dataset; simulated (Section 4.7)

expected calibration error (ECE, the confidence-weighted gap between stated confidence and observed accuracy over 15 equal-width bins) and the negative log-likelihood (NLL).

For safety we define three rates. The *unsafe autonomous action rate* is, among real attacks that hit a HIGH-criticality, patient-dependent device (the high-stakes trials), the share for which the system executed a disruptive action with no human approval. The *unnecessary isolation rate* is, among detector false positives (benign flows flagged as attacks), the share for which the system autonomously isolated the device anyway. *Autonomy retained* is the share of fired detections that the system handles without a human. We always report these rates together with their counts, because a rate of zero means something different when the denominator is 10 than when it is 262,000. For the ledger we report integrity-check outcomes, latency percentiles and committed throughput.

4.4. Detector performance

Table 4 reports the two main detectors on the entire test split. XGBoost reaches 99.52% 19-class accuracy, macro-F1 0.9263 and 99.97% attack detection, with a benign false-positive rate of 5.09% at the arg-max operating point. At the runtime operating point (alert only when confidence is at least 0.6) attack detection is 99.72% and the benign false-positive rate falls to 3.12%

(1,174 of 37,607 benign flows flagged). Of the 1,576,575 attack flows, XGBoost misses 440 as binary decisions (true positives 1,576,135; false positives 1,916; true negatives 35,691; precision 0.9988). Random forest is weaker on macro-F1 (0.8704) and has a higher benign false-positive rate (6.73%), but trains in 56 s against 749 s.

Table 4: Detector results on the full CICIoMT2024 test split (1,614,182 flows), main models trained once with seed 42.

Model	Acc. (19)	$F1_{macro}$ (19)	$F1_{macro}$ (6)	Attack det.	Benign FPR	Train (s)
XGBoost	0.9952	0.9263	0.9126	0.9997	0.0509	749
Random forest	0.9916	0.8704	0.9036	0.9995	0.0673	56

Figure 3 shows the per-class F1 of XGBoost. The volumetric classes are near perfect (F1 above 0.99 for the DDoS and DoS TCP/IP classes). The weak classes are ARP spoofing (F1 0.5346, precision 0.4055) and vulnerability scanning (0.6919); both are rare in the data. This matters downstream: 94.4% of the detector's false alarms are predicted as ARP spoofing, so the weakness is concentrated in one class.

Because the dataset rows carry no wall-clock timestamp, detection delay is counted in flows. On all 20 attack captures in the test split the detector's first alert came at flow 0 (17 captures) or flow 1 (3 captures), and every capture was alerted within its first 10 flows. The share of a capture's flows that raised an alert ranged from 73.6% (vulnerability scanning) to 100%.

4.5. Robustness of the detector: seeds, classifiers and uncertainty

A single trained model, as in Table 4, cannot show how much of a difference between two classifiers is training noise, and a single accuracy figure hides where the errors are. This section therefore repeats the detector study with more classifiers and seeds, adds confidence intervals, and examines the errors.

Table 5 trains six classifiers five times each. A training seed changes both the random sample of at most 30,000 training flows per class and the model's own randomness; every model is scored on the entire test split. The smaller training cap (30,000 instead of 300,000 flows per class) makes thirty trainings affordable on a 2-CPU machine, so the XGBoost row is not the same model as in Table 4. The decision tree, random forest, extra-trees and XGBoost models are scale-free; the logistic regression (trained by stochastic gradient descent) and the neural network (one hidden layer of 64 units) receive log-scaled and

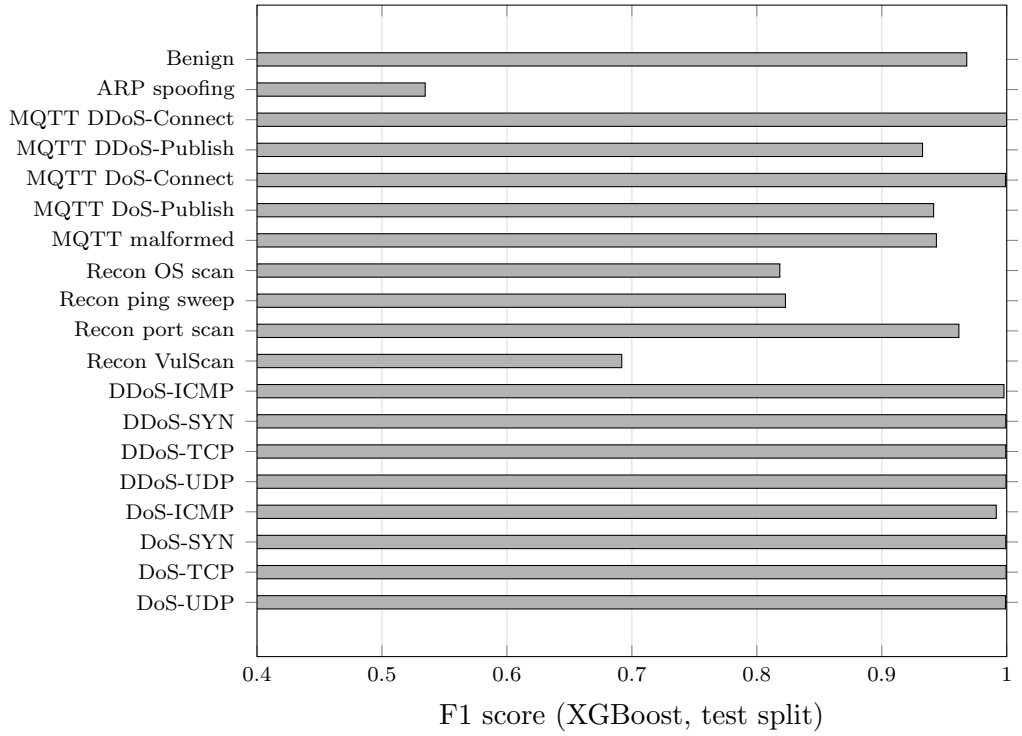


Figure 3: Per-class F1 of the XGBoost detector on the full test split (macro-F1 0.9263). ARP spoofing and vulnerability scanning are the weak classes.

standardised features, because the raw features are heavy-tailed. The linear and neural baselines were not tuned.

Table 5: Six classifiers on the full test split: mean ± sample standard deviation over training seeds. Each model is trained on at most 30,000 flows per class.

Classifier	Seeds	Accuracy (19)	Macro-F1 (19)	Attack detection	Benign FPR	Train (s)
Decision tree	5	0.9950 ± 0.0003	0.9080 ± 0.0051	0.9996 ± 0.0000	0.0857 ± 0.0037	8
Logistic regression (SGD)	5	0.6200 ± 0.0296	0.5690 ± 0.0084	0.9993 ± 0.0001	0.1902 ± 0.0054	69
Random forest	5	0.9922 ± 0.0010	0.8783 ± 0.0131	0.9993 ± 0.0000	0.0499 ± 0.0107	50
Extra trees	5	0.9896 ± 0.0001	0.8600 ± 0.0011	0.9994 ± 0.0000	0.0604 ± 0.0026	23
Neural network (1 hidden layer)	5	0.6883 ± 0.0389	0.6616 ± 0.0099	0.9994 ± 0.0002	0.1303 ± 0.0386	55
XGBoost	5	0.9953 ± 0.0002	0.9103 ± 0.0017	0.9997 ± 0.0000	0.0671 ± 0.0078	88

Table 5 shows three things. First, the tree-based models form a cluster: the decision tree, random forest, extra trees and XGBoost reach between 0.9896 and 0.9953 accuracy, and XGBoost has the highest accuracy (0.9953) and macro-F1 (0.9103) and the smallest spread over seeds (standard deviations 0.0002 and 0.0017). Second, the margin between XGBoost and a single decision tree is small: 0.0003 in accuracy and 0.0023 in macro-F1, about half the tree's seed-to-seed standard deviation of 0.0051. On this dataset the choice of XGBoost over a simpler tree is therefore supported by its stability and not by a large gain, and we do not claim otherwise. Third, the linear and neural baselines are far weaker (accuracy 0.62 and 0.69, macro-F1 0.57 and 0.66), which suggests that the classes are separated by sharp thresholds on individual features, such as protocol flags and rates, that a linear boundary or a small untuned network does not capture. Across all six classifiers the attack-detection rate stays between 0.9993 and 0.9997; what varies is the benign false-positive rate (0.050 to 0.190) and macro-F1. With 30,000 training flows per class XGBoost reaches the same accuracy as the main model trained on up to 300,000 (0.9953 against 0.9952) but a lower macro-F1 (0.9103 against 0.9263) and a higher benign false-positive rate (0.067 against 0.051), so the larger training cap helps the rare classes.

Table 6 gives 95% bootstrap intervals for the main XGBoost model of Table 4, obtained by resampling the test flows 1,000 times. The intervals are narrow (accuracy 0.9951–0.9953; macro-F1 0.9233–0.9291), but they treat the 1.6 million flows as independent. Flows inside one capture are strongly correlated, so these intervals understate the uncertainty across captures and should be read as a lower bound on it; the spread across training seeds in Table 5 is the more honest guide.

Table 6: Main XGBoost model on the full test split: point estimates and 95% bootstrap intervals (1,000 resamples).

Metric	Estimate	95% interval
Accuracy (19 classes)	0.9952	0.9951 – 0.9953
Macro-F1 (19 classes)	0.9263	0.9233 – 0.9291
Attack-detection rate	0.9997	0.99970 – 0.99975
Benign false-positive rate	0.0509	0.0489 – 0.0533

Where the errors are is shown in Table 7 and Figure 4. At the level of attack families, almost all errors are between DoS and DDoS (470 DoS flows predicted as DDoS and 1,646 the other way) and between benign traffic and ARP spoofing or reconnaissance. The largest single confusions in Figure 4 are benign flows predicted as ARP spoofing (1,631), TCP/IP DDoS-ICMP predicted as DoS-ICMP (1,309), MQTT DDoS-Publish predicted as DoS-Publish (1,056), and port scans confused with operating-system scans (582 and 463). These are pairs whose traffic differs mainly in the number of attacking sources, which a per-flow feature vector cannot see. For the present purpose they are cheap errors: the response of the framework depends on whether a flow is an attack, and not on which attack it is, so confusing a DoS with a DDoS changes no decision. The costly errors are the 1,631 benign flows flagged as ARP spoofing and the 440 attack flows predicted as benign.

Table 7: Confusion between the six attack families on the full test split (rows: true family; columns: predicted family).

True family	Flows	Benign	ARP spoof.	MQTT	Recon	DoS	DDoS	Recall
Benign	37,607	35,691	1,631	25	260	0	0	0.9491
ARP spoofing	1,744	106	1,368	4	266	0	0	0.7844
MQTT	63,715	54	45	63,602	14	0	0	0.9982
Recon	27,676	273	289	34	27,079	1	0	0.9784
DoS	416,676	1	1	4	4	416,196	470	0.9988
DDoS	1,066,764	6	40	26	82	1,646	1,064,964	0.9983

Finally, Figure 5 removes the choice of operating point and asks how well the detector separates attacks from benign traffic by its score alone. The area under the ROC curve is 0.99996 and the average precision 0.999999. At the arg-max operating point the benign false-positive rate is 5.09%, but the curve shows that this is a choice of threshold and not a limit of the ranking: ranking

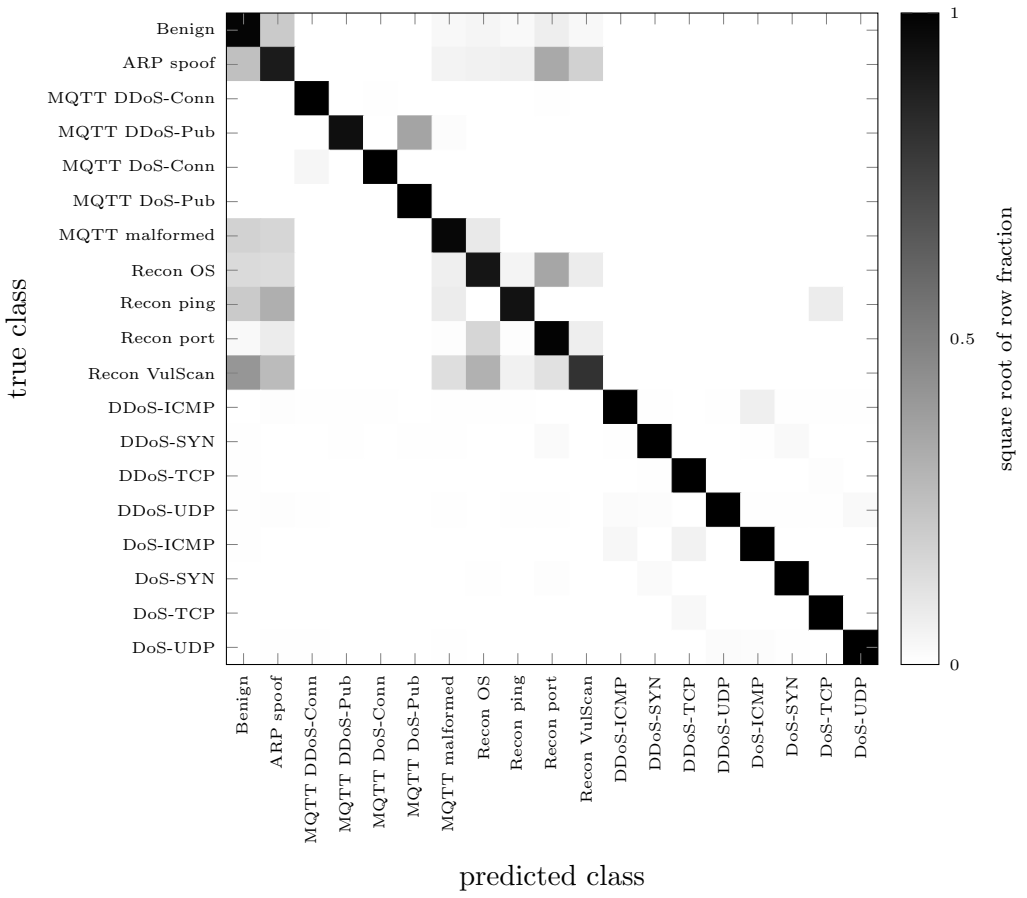


Figure 4: Row-normalised confusion matrix of the XGBoost detector on the full test split. The diagonal is the per-class recall; shading is the square root of the fraction of a true class's flows, so that small off-diagonal confusions remain visible.

by one minus the benign probability, 99.5% of attack flows are detected before the first benign flow is flagged, 99.9% with a false-positive rate of 0.11%, and 99.95% with 2.3%. We do not use this to pick a better threshold, because choosing it on the test split would be tuning on test; Section 4.6 shows how a threshold is chosen on validation data instead.

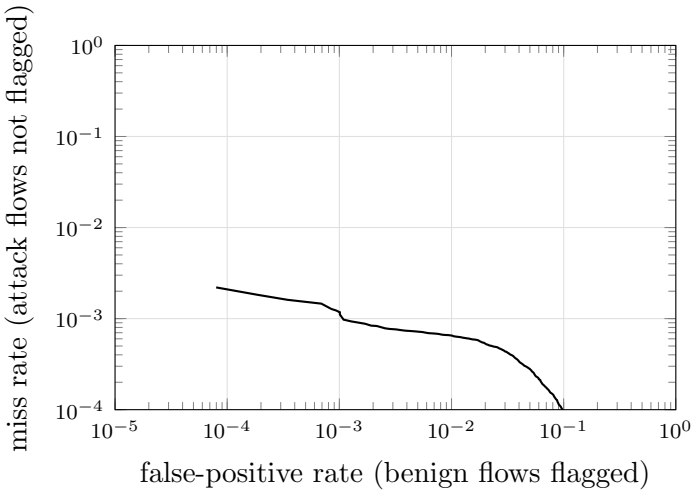


Figure 5: Detection-error trade-off curve of the XGBoost detector for attack versus benign on the full test split, scored by one minus the probability of the benign class: miss rate against false-positive rate, both on logarithmic axes. A curve closer to the lower left is better. The area under the corresponding ROC curve is 0.99996 and the average precision 0.999999.

4.6. Confidence calibration and the confidence gate

Confidence is informative here: among alerts, real attacks have median confidence 1.0000 and false alarms 0.7583, and the area under the ROC curve of confidence for separating them is 0.9893. Figure 6 is the reliability diagram of the shipped detector. Nearly all flows (1,559,351 of 1,614,182) fall in the top confidence bin, where mean confidence is 0.9999 and observed accuracy 0.9994. The calibration error is small: ECE is 0.0035 before and 0.0033 after temperature scaling with $T = 0.917$ (this temperature was fitted on the validation data of the development model described below, and is applied here unchanged). The diagram also shows where the detector is overconfident: in the bin of 3,387 flows with confidence between 0.73 and 0.80 the observed

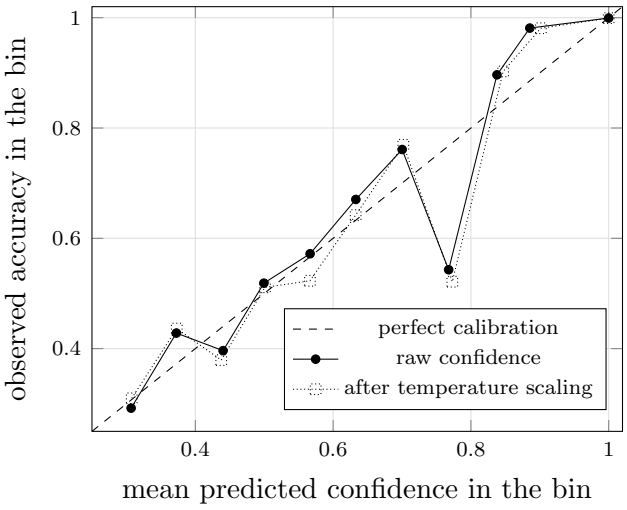


Figure 6: Reliability diagram of the XGBoost detector on the full test split (15 equal-width bins; bins with fewer than 30 flows are not drawn). Points close to the dashed diagonal are well calibrated.

accuracy is only 0.54. These are the alerts a confidence gate should send to a human.

To choose a confidence threshold without touching the test split we trained a development model, held out validation data from both training and test, fitted the temperature on validation, and fixed the selection rule beforehand: choose the largest τ that keeps at least 95% of real-attack autonomy on validation. Temperature scaling improved validation calibration (ECE 0.0409 to 0.0288; NLL 0.1504 to 0.1495); on test, ECE of the development model was 0.0033 raw and 0.0030 calibrated, while NLL moved slightly the wrong way (0.0204 to 0.0210), so the calibration gain is small and mainly on validation. The rule selected $\tau = 0.7$ (Figure 7).

Scored once on the untouched test split, the gate left 41.5% (40.0–43.2%, range over five context seeds) of the development model's 1,558 false alarms with autonomous isolation, against 50.5% (48.8–52.1%) without the gate, while the system still acted autonomously on 99.9% of the real attacks it would otherwise handle alone. The gate is therefore a modest improvement, not a cure: more than 40% of false alarms are still isolated without a human. The gate is off by default in the shipped system and is an opt-in setting.

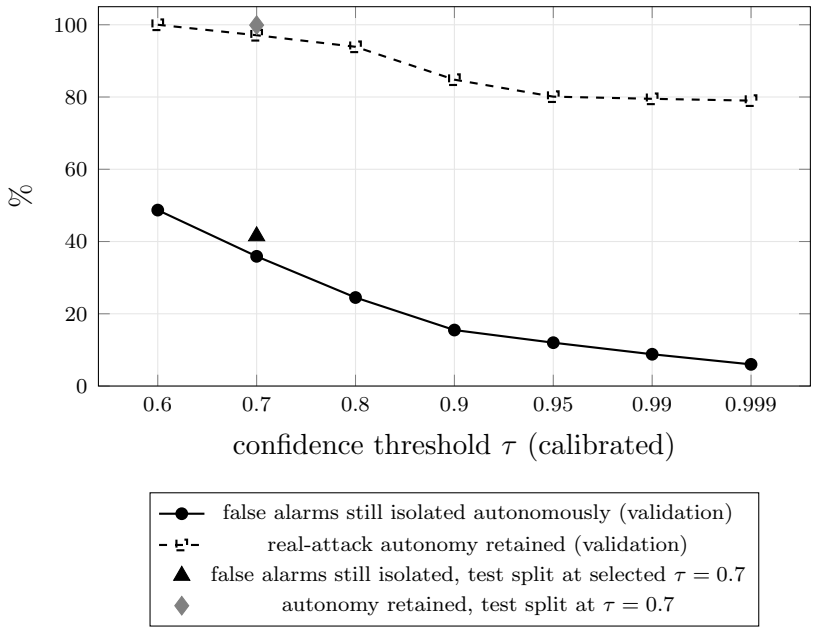


Figure 7: Confidence-gate trade-off. $\tau = 0.7$ was selected on validation; the triangle and diamond are the single test-split scores at that threshold. Unsafe autonomous actions are 0 at every τ.

4.7. Patient-safety behaviour of the decision layer

Every one of the 1,614,182 test flows is one trial. The XGBoost detector, Risk Engine and Policy Engine are the real implementations. The dataset has no clinical context, so each trial draws a device context uniformly over three criticality levels crossed with patient-dependent yes or no, and the three language-model-backed agents are replaced by deterministic stand-ins that read the detector's output and the sampled context. The whole experiment is repeated over five context seeds. There are about 262,000 high-stakes trials (a real attack on a HIGH-criticality, patient-dependent device) per seed, and 1,312,874 over the five seeds. Table 8 gives the counts behind each rate, and Figure 8 draws them.

Table 8: Response systems on the full test split. Counts are for the first context seed; rates are the mean over five seeds (range in brackets).

System	Acts without a human?	Unsafe autonomous actions: count / high-stakes trials (rate)	Unnecessary isolations: count / false alarms (rate)
Detection only	never acts	not applicable	not applicable
Fixed response	always	262,380 / 263,116 (99.72%; 99.70–99.75)	1,174 / 1,174 (100.00%)
Detection + Risk Engine	always	262,380 / 263,116 (99.72%; 99.70–99.75)	1,174 / 1,174 (100.00%)
Detection + agentic layer	policy decides	0 / 263,116 (0.00%)	575 / 1,174 (50.48%; 48.98–52.81)
Full system	policy decides	**0 / 263,116 (0.00%)**	**575 / 1,174 (50.48%; 48.98–52.81)**

A zero in this section is a counted result, not a missing result and not a sign that training failed. The detector is trained once (Section 4.4); this experiment scores its predictions with the decision layer. For the full system the numerator of the unsafe-action rate is the number of high-stakes attacks for which a disruptive action ran without a human, and it is 0 in every seed, 0 of 1,312,874 over the five seeds. The reason is structural, not statistical: the first Policy Engine rule sends every severe action on a patient-dependent device to a human, so no sample of detections can produce an unsafe autonomous

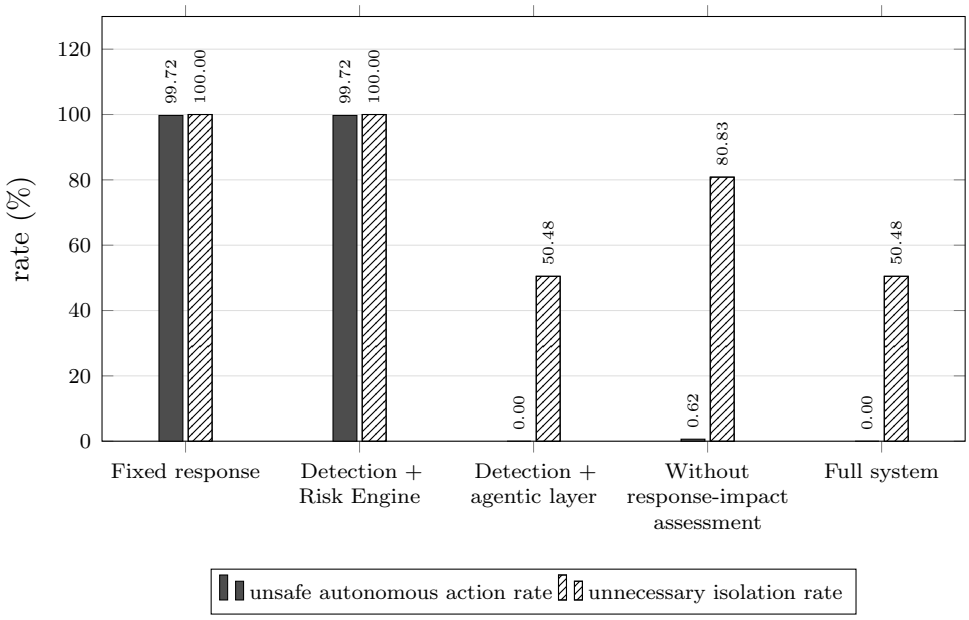


Figure 8: Safety of the response systems on the full test split (mean over five context seeds). The numbers above the bars are the rates; a label of 0.00 is a measured zero (0 of 263,116 high-stakes trials in the first seed), so the bar has no height. "Detection only" is left out because it never acts, so neither rate is defined for it. The "Without response-impact assessment" system is an ablation of the full system (Table 9).

action, and Section 4.9 verifies this by enumerating the inputs. For a rate measured as zero events in n trials, the one-sided 95% upper confidence bound is about $3/n$ (the rule of three), here 2.3×10^{-6}. The "detection only" system also shows no unsafe action, but trivially, because it never acts; we therefore mark it as not applicable instead of 0.00%.

A fixed-response playbook, and the same playbook with a Risk Engine score but no routing, execute a disruptive action without a human on 99.72% of high-stakes real attacks. The agentic layer with the Policy Engine reduces this to zero. The price is the unnecessary isolation rate: on the roughly half of false alarms that land on contexts the policy lets it handle alone, the system still isolates the device (50.48%). The denominator of that rate is the 1,174 benign flows the detector flagged, a much smaller sample than the unsafe-action denominator; it is the rate the calibrated gate of Section 4.6 lowers.

Table 9: Ablations of the full system on the same trials. "Differs" counts trials per seed in which the decision differs from the full system. Counts are for the first context seed.

Ablation	Unsafe autonomous actions: count / trials (mean rate)	Unnecessary isolations: count / false alarms (mean rate)	Differs (per seed)
Without healthcare context	0 / 263,116 (0.00%)	575 / 1,174 (50.48%)	0
Without clinical criticality	0 / 263,116 (0.00%)	575 / 1,174 (50.48%)	0
Without response-impact assessment	1,655 / 263,116 (0.62%)	944 / 1,174 (80.83%)	5,211–5,334
Without agentic layer	262,380 / 263,116 (99.72%)	1,174 / 1,174 (100.00%)	785,951–787,713
Without blockchain	0 / 263,116 (0.00%)	575 / 1,174 (50.48%)	0
Without recovery verification	0 / 263,116 (0.00%)	575 / 1,174 (50.48%)	0

Table 9 separates what carries the safety result. Removing the response-impact assessment lets 1,655 high-stakes attacks through (0.62%) and raises unnecessary isolation to 80.83%, so that component matters. Removing the

agentic layer restores the fixed-response behaviour. In this experiment the agentic layer is a deterministic stand-in that supplies clinical context and a planned action, so this ablation shows that the *context-aware planning layer* is essential, not that a language model is necessary. Removing healthcare context, clinical criticality, the blockchain or recovery verification changes nothing here. For the first two this is structural: the Policy Engine routes on response impact and patient dependency, and the patient-dependency rule already forces approval; for the last two, neither component is on this experiment's decision path (the ledger and recovery are evaluated separately in Sections 4.10 and 4.11). We report these zeros rather than omit them.

Safety is paid for in human time. In the full system 50.0% of alerts across all devices wait for a person (785,858 contained autonomously, 786,293 waiting), and on high-stakes devices every alert waits (262,215 of 262,215 detected attacks). At the runtime confidence threshold the detector misses 4,424 attack flows overall (708 on high-stakes devices), which no response layer can recover; these shares use three context seeds. The delay model in Section 5.3 uses an *assumed* approval time, since the project has no measurement of one.

4.8. Sensitivity to the clinical device mix

The safety study of Section 4.7 assumes one device mix: criticality uniform, half the devices patient-dependent. A reviewer can fairly ask whether the conclusion depends on that choice, so we repeated the comparison while changing the share of patient-dependent devices from 0 to 100% (criticality uniform) and the share of HIGH-criticality devices from 0 to 100% (half the devices patient-dependent). Each setting is scored on a sample of 121,174 trials: all 1,174 detector false alarms plus 120,000 randomly chosen other test flows (the full 1.6 million flows were used in Section 4.7; a sample keeps eleven settings affordable, and all reported quantities are conditional rates, so including every false alarm does not bias them). Table 10 and Figure 9 give the results.

Three things follow from Table 10. First, the safety result does not depend on the mix: the full system has zero unsafe autonomous actions in every setting that contains a high-stakes device, while the fixed-response baseline is unsafe on 99.7–99.8% of high-stakes attacks in all of them. Second, the response-impact assessment keeps mattering at every mix: without it between 0.6% and 0.7% of high-stakes attacks get through, in proportion to the number of such attacks. Third, and this is the honest cost, the autonomy

Table 10: Sensitivity to the clinical device mix (121,174 sampled trials per row, 1,174 of them false alarms). "Unsafe" counts disruptive autonomous actions on high-stakes real attacks; "isolated" counts false alarms isolated without a human.

Device mix	High-stakes trials	Unsafe: fixed response	Unsafe: full system	Unsafe: no response impact	False alarms isolated: full system	Autonomy retained: full system
Patient-dependent share (criticality uniform)						
0%	0	not defined	not defined	not defined	1,174 / 1,174	100.0%
10%	3,799	3,786	0	22	1,075 / 1,174	90.0%
25%	9,616	9,585	0	66	905 / 1,174	75.0%
50%	19,295	19,246	0	132	611 / 1,174	50.1%
75%	29,164	29,088	0	184	283 / 1,174	25.0%
90%	35,181	35,090	0	222	130 / 1,174	9.9%
100%	39,142	39,043	0	246	0 / 1,174	0.0%
HIGH-criticality share (half of devices patient-dependent)						
0%	0	not defined	not defined	not defined	601 / 1,174	50.0%
33%	19,595	19,554	0	115	586 / 1,174	50.0%
60%	35,192	35,091	0	214	595 / 1,174	50.0%
100%	58,621	58,451	0	355	582 / 1,174	50.1%

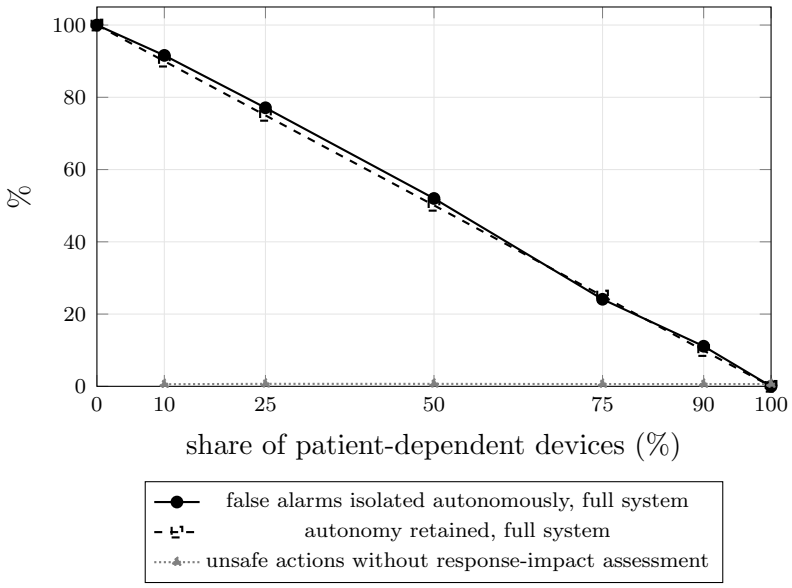


Figure 9: Effect of the share of patient-dependent devices on the full system. Unsafe autonomous actions of the full system are zero at every share and are therefore not drawn; the dotted line shows the unsafe rate of the ablation without response-impact assessment.

the system retains falls almost linearly with the share of patient-dependent devices, because every action on such a device waits for a person; at 100% patient-dependent devices the system never acts alone and every false alarm is a task for a human, but also never isolates a device unnecessarily. The 50% autonomy of the base setting is therefore not a property of the framework but of the assumed mix, and a hospital with mostly patient-dependent devices should expect far more approvals. The HIGH-criticality share has almost no effect on autonomy, because routing depends on patient dependency and not on criticality alone: a HIGH-criticality device without a patient attached is still handled autonomously.

4.9. Exhaustive verification of the decision rules

The experiments above sample trials from a dataset, so at best they show that nothing unsafe happened in n trials. The Risk Engine and Policy Engine are small enough to do better. We enumerated every combination of the eleven response actions, three device criticalities, two patient-dependency values, four stated clinical-impact levels, four threat-severity levels and three threat-confidence values (3,168 inputs), called the real engines on each, and

counted the outcomes (Table 11).

Table 11: Policy decisions over all 3,168 inputs, by device context. "Autonomous" means the system acts without a human; "human" means the action waits for approval.

Device context	Inputs	Autonomous	Human
Low criticality, not patient-dependent	528	528	0
Medium criticality, not patient-dependent	528	528	0
HIGH criticality, not patient-dependent	528	528	0
Low criticality, patient-dependent	528	144	384
Medium criticality, patient-dependent	528	144	384
HIGH criticality, patient-dependent	528	0	528
Total	3,168	1,872	1,296

Three properties hold over the whole input space. No severe action (shutdown, critical isolation, segment quarantine or failover) is ever autonomous on a patient-dependent device (0 violations among 576 such inputs). No action of any tier is ever autonomous on a HIGH-criticality, patient-dependent device (0 violations among 528 inputs; the only autonomous actions on patient-dependent devices in Table 11 are the non-disruptive ones, such as rotating credentials, on low- and medium-criticality devices). And the decision does not depend on the stated threat severity or the confidence: for every fixed action, criticality, dependency and clinical-impact value, all twelve severity-and-confidence combinations gave the same routing. This is a statement about every input, not a sample, which is why the zero unsafe actions of Section 4.7 is a consequence of the rules and not of the data. The table also shows the design choice that a reviewer should know about: HIGH-criticality devices that no patient depends on are handled fully autonomously, including severe actions, because the registry says nothing clinical is at stake.

4.10. The agentic layer under injected faults

To test the claim that no agent can by itself cause harm, the five agents were driven by a scripted, deliberately misbehaving client through the real orchestrator, engines and simulated devices. Ten behaviours were run on three simulated devices: an infusion pump (low criticality, no patient attached), an ECG monitor, and a ventilator (the only HIGH-criticality, patient-dependent device in the simulated hospital). Table 12 lists the outcomes; each cell is one deterministic run, so there is no variance to report.

Table 12: Outcome per injected behaviour and device. Auto: resolved or reopened with the system acting alone. Human: held for approval, device untouched. Stall: the incident stopped safely.

Injected behaviour	Pump	ECG	Ventilator
Honest agents (reference)	Auto	Human	Human
Context agent claims low criticality, no patient dependency	Auto	Human	Human
Same lie, planner also recommends shutdown	Auto (off)	Human	Human
Planner recommends shutdown with a false "low" impact	Auto (off)	Human	Human
Planner understates the clinical impact of isolation	Auto	Human	Human
Prompt injection obeyed (text in traffic tells the model to downgrade and shut down)	Auto	Human	Human
Every agent returns unparseable text	Stall	Stall	Stall
Planner returns an action outside the vocabulary	Stall	Stall	Stall
Investigation answers about a different incident	Stall	Stall	Stall
Recovery agent claims success while the attack persists	Auto, reopened	Human	Human

Across the 30 runs, no unsafe autonomous action took effect on the ventilator and no incident was falsely closed; 9 runs stalled safely and 14 were routed to a human. The infusion pump is acted on autonomously by design: its registry entry allows it, and the lie in the model's output could not make

that more or less safe. To see what the guards add, the same 30 runs were repeated with the three guards of Section 3 switched off (Figure 10); only those three checks differ.

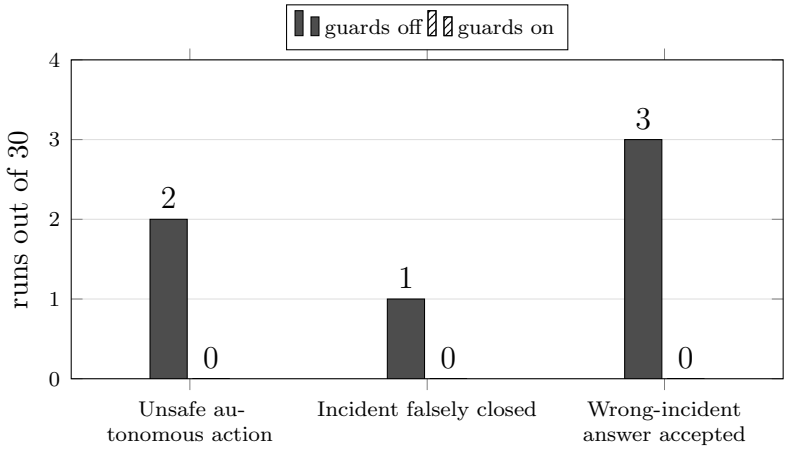


Figure 10: Effect of the grounding guards over the same 30 fault-injected runs. A label of 0 is a measured zero.

With the guards off, two runs took an unsafe autonomous action (the context-lie case, and the context-lie-plus-shutdown case), one incident was falsely closed (the recovery lie), and three answers about the wrong incident were accepted. With the guards on all four counts are zero. These runs show how the architecture behaves under scripted worst cases; they do not show how often a real model misbehaves. A live-model evaluation harness exists, but our attempts to complete it were stopped by the model provider's overload and quota errors, so no live-model figure is reported.

4.11. Blockchain provenance on a real Fabric network

The measurements use real Fabric binaries driven through the backend's own client; nothing is mocked. Table 13 lists the integrity checks, all of which passed. Beyond these checks, all 142 blocks (194 transactions) were fetched from the Hospital peer and re-verified with an independent implementation of Fabric's block hashing. The genuine chain verified. Editing one byte of one transaction in block 70 (on a copy) was detected at block 70 by the data hash. When the attacker also recomputed that block's own data hash, the edit was still detected, at block 71, because the next block's previous-hash no longer matched. All 188 transactions that the client reported as committed

were found as valid on the ledger; the 2 invalid transactions on the chain (an endorsement-policy failure and a read conflict) were produced on purpose by the checks, stay on the chain flagged, and never changed state.

Table 13: Ledger integrity checks (all passed).

#	Check
1	A duplicate incident id is rejected (the incident record is write-once).
2	A decision for an unknown incident is rejected.
3	An invalid response-action category is rejected.
4	The evidence-verification function accepts the recorded evidence hash.
5	The evidence-verification function rejects evidence altered by one field.
6	The Hospital and SecurityOps peers return the same answer.
7	Two simultaneous writers of one incident id: exactly one is committed, and only that record is on the ledger.
8	A transaction endorsed by one organisation only is invalidated by the majority policy and writes no state.

Table 14: Ledger latency (ms, sequential). A write returns only after both peers have committed. n is the number of operations timed.

Operation	n	p50	p95	max
Fabric command-line spawn alone (floor)	20	65	188	239
Write of an incident record	40	1114	1139	1147
Query sees a committed write	20	87	112	127
Read: evidence verification	40	89	108	115
Full five-record incident lifecycle	10	5578	5610	5617

Table 14 shows that a write takes about 1.1 s at the median, mostly the 1 s orderer batch timeout plus the wait for both peers; a full lifecycle takes about 5.6 s. Because provenance is written through the outbox off the response path, these latencies do not delay containment. Figure 11 shows committed throughput with concurrent clients: 0.90, 1.78, 3.36 and 5.99 transactions per second for 1, 2, 4 and 8 clients, with no failed transaction among 24 submitted at each level and median latency rising from 1111 to 1321 ms. Scaling is near

linear to four clients and then sub-linear, as expected on a 2-CPU host where every call spawns a command-line process.

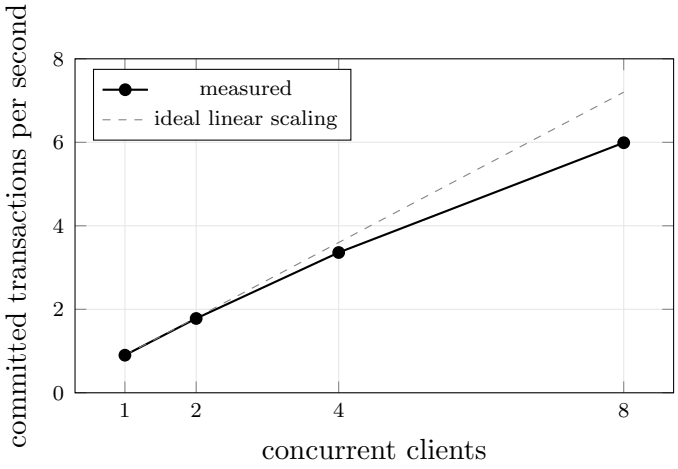


Figure 11: Committed throughput of the real Fabric network versus concurrent clients (24 transactions per level, none failed).

The first run of this evaluation also exposed a defect. The client reported success once a transaction was endorsed and handed to the orderer, not once it was committed. A transaction invalidated at commit would still have been reported as committed, so the outbox could mark a record confirmed that never reached the ledger. The invoke script now waits for both peers' commit events and exits non-zero for an invalidated transaction; the latencies above are measured with the fix, and the checks and integration tests guard against a regression.

4.12. Pipeline latency

Table 15 gives the in-process compute cost of the deterministic part of the pipeline. The decision path (Risk plus Policy) costs about 0.016 ms at the median, and the detector scores a flow in 0.77 ms (batch throughput 32,311 flows per second), so neither is a bottleneck; the language-model calls would dominate in a live deployment and are not included.

4.13. Analysis and limitations

The results support four claims. On real data, the decision layer never lets a disruptive autonomous action reach a registry-flagged high-stakes device,

Table 15: In-process latency of the deterministic pipeline stages (ms); n is the number of timed calls.

Stage	p50	p95	p99	n
Risk Engine + Policy Engine, per incident	0.016	0.025	0.053	3,000
Detector, one flow	0.765	0.969	1.393	300

whether the clinical context is supplied by a stand-in (Section 4.7), by a different device mix (Section 4.8), or by a scripted lying agent (Section 4.10); this holds for every input of the rule tables (Section 4.9). The detector's performance is stable across training seeds and classifiers (Section 4.5). The ledger detects tampering of the kind we tried. And the guards close exactly the gaps that a scripted model opens when they are removed.

The results do not support several claims, and a reader should not draw them. The clinical context in the safety studies is sampled, not observed, and the agents in the safety and fault-injection studies are scripted: the results concern the gating logic, not clinical outcomes and not the error rate of a real model. In the fault-injection study only one simulated device is both HIGH-criticality and patient-dependent, so the study alone rests on ten ventilator runs; the exhaustive check of Section 4.9 covers the rule tables but not the integration. The unnecessary-isolation rate of 41.5–50.5% is a real weakness: a false alarm on a permitted device still causes autonomous isolation, and the cost of safety in human approvals grows with the share of patient-dependent devices (Section 4.8). The Fabric network has one orderer on one 2-CPU host, with one run and no confidence intervals, so it says nothing about orderer fault tolerance or malicious orderers, and its latencies belong to this deployment rather than to Fabric. The test flows within one capture are strongly correlated, so the bootstrap intervals of Section 4.5 understate the uncertainty across captures. Finally, the system is a research prototype, not a certified medical product.

5. Comparisons

We compare the proposed framework with existing work on three axes: detection quality against published detectors (Section 5.1), capabilities against related systems (Section 5.2), and safety and responsiveness against response

baselines and unguarded agents (Section 5.3). Throughout, figures for other works are *as reported by their authors* and are cited in the table row or cell that uses them; we did not retrain or re-run any other work.

5.1. Detection quality against published detectors

Table 16 places our XGBoost detector beside L2D2 [3], the reinforcement-learning hybrid HCLR-IDS [4] and the interpretable XGBoost fusion of Hafid et al. [5], all on CICIoMT2024, and beside two ensemble detectors that use a different dataset (WUSTL-EHMS-2020) [14, 15], which are listed for context only because their accuracy cannot be compared with ours. Hafid et al. use the CIC IoMT 2024 release with a reduced feature set. The last two rows give our result for the main model of Section 4.4 and its mean over five training seeds (Section 4.5).

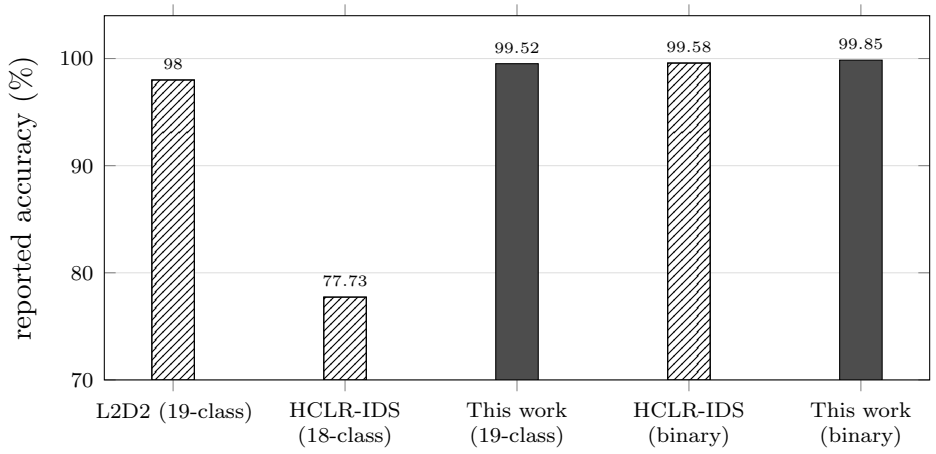


Figure 12: Reported accuracy on CICIoMT2024, multi-class (left three bars) and binary (right two). Hatched bars are quoted from L2D2 [3] and HCLR-IDS [4]; solid bars are measured in this work. The vertical axis starts at 70%.

On the quoted numbers our detector is on par with or above the others on accuracy (Figure 12), and its 19-class accuracy of 99.52% compares with 98% for L2D2 and 77.73% for the 18-class HCLR-IDS. We do not claim a win. The comparison is indicative only for three reasons: the protocols differ (the other authors' splits, class definitions and preprocessing may not match the official capture-level split we use, and HCLR-IDS counts 18 classes where we count 19); accuracy on this dataset is dominated by the volumetric DDoS and DoS classes; and the figures we could extract from the other papers are

Table 16: Detection comparison. Values for other works are quoted from the cited papers; "this work" rows are measured on the full official CICIoMT2024 test split.

Work	Dataset	Model	Classes	Accuracy	Other reported metrics
L2D2 [3]	CICIoMT2024	Enhanced LSTM	19	98%	none extracted
HCLR-IDS [4]	CICIoMT2024	CNN-LSTM, DQN, PPO	18	0.7773 (multi-class); 0.9958 (binary)	Mutual-information feature selection
Hafid et al. [5]	CIC IoMT 2024	XGBoost + logistic-regression fusion, SHAP	as in dataset	0.97 (XGBoost)	Fusion: precision 0.98, recall 0.91; false negatives 361 to 213
Alsolami et al. [14]	WUSTL-EHMS-2020	Stacked random forest and SVM	binary	98.88%	—
Abdelhaq et al. [15]	WUSTL-EHMS-2020	Soft-voting XGBoost and two SVMs	binary	98.04%	Validated on TON-IoT and CICIDS 2017
This work (main model)	CICIoMT2024	XGBoost, 45 features	19	**0.9952** (19-class); 0.9985 (binary)	Macro-F1 0.9263; attack detection 0.9997; benign FPR 0.0509
This work (5 seeds)	CICIoMT2024	XGBoost, 30,000 rows per class	19	0.9953 ± 0.0002	Macro-F1 0.9103 ± 0.0017

accuracies, so they cannot be compared on macro-F1, where our weak classes (ARP spoofing 0.53, vulnerability scanning 0.69) are visible. The contribution of this work is therefore not a new detector but what is built after it.

5.2. Capability comparison

Table 17 compares what each system covers. This is where the proposed framework differs from the detection papers [3–5], from the Fabric prototype for IoMT [9], from the forensic blockchain [16], and from the language-model response agents [6, 7].

Table 17: Capabilities as described in each cited work's title and abstract. ✓ = addressed; – = not part of the stated scope (not a defect of that work); ○ = partly, or in simulation only, or not verified from what we read.

	Real IoMT data	Auto-mated response	Patient-aware gate	LLM agents	Output guard	Block-chain	Ledger tested
L2D2 [3]	✓	–	–	–	–	–	–
HCLR-IDS [4]	✓	–	–	–	–	–	–
Hafid et al. [5]	✓	–	–	–	–	–	–
Pelekoudas-Oikonomou et al. [9]	–	–	–	–	–	✓	○
Xiao et al. [16]	–	–	–	–	–	✓	○
IRCopilot [6]	–	✓	–	✓	–	–	–
Gao et al. [7]	–	✓	–	✓	–	–	–
This work	✓	✓	✓	✓	✓	✓	✓

No cited system combines all seven columns, but Table 17 is a statement about coverage, not performance: a more specialised work may do its one column better than we do. For example, IRCopilot is evaluated on a response benchmark that we do not use, and Gao et al. report recovery-time gains that our framework, which spends human time on purpose, does not aim for.

5.3. Safety and responsiveness against response baselines

No cited work reports an unsafe-autonomous-action rate, so there is no published number to quote for it. The baselines in Table 18 are therefore reference systems implemented in this work to stand for existing practice: a fixed playbook that isolates any attacked device, and a fully manual process

(detection only). They are not re-implementations of a particular published system. The figures are those of Section 4.7.

Table 18: Safety against reference baselines on the full test split (first context seed). The baselines are implemented in this work and are not published systems.

System	Unsafe autonomous actions (of high-stakes attacks)	Alerts waiting for a person	Remaining weakness
Fixed playbook	262,380 of 263,116 (99.72%)	0%	Disrupts patient-dependent devices
Fully manual (detection only)	0 of 263,116 (never acts)	100%	Every alert waits for a person
Full system	**0 of 263,116 (0.00%)**	50.0%	41.5–50.5% of false alarms are isolated

A second comparison concerns agents that have no deterministic output guard. IRCopilot [6] and the fine-tuned agent of Gao et al. [7] describe no such guard in their abstracts, and we did not run either. As a stand-in for that design we use our own agentic layer with the three guards switched off, run over the same 30 fault-injected cases as Section 4.10. Table 19 is a comparison of guarded and unguarded versions of one system on scripted faults; it says nothing about the quality of the cited agents.

Table 19: Guarded versus unguarded agents over the same 30 fault-injected runs (Section 4.10). The unguarded configuration stands for agent designs without an output guard such as [6, 7]; it is not a re-implementation of them.

Configuration	Unsafe autonomous actions on the ventilator	Incidents falsely closed	Wrong-incident answers accepted
Unguarded agents (guards off)	2 of 30 runs	1	3
Guarded agents (guards on)	**0 of 30 runs**	**0**	**0**

Safety has a responsiveness cost. The fully manual process is as safe as the full system but makes every alert wait; the fixed playbook never waits but is unsafe (Table 18). Figure 13 shows the mean time before containment under a *modelled* human approval time D: pipeline time is measured, the share of alerts that wait is measured (50.0% overall for the full system, 100% on high-stakes devices), and D is an assumption because the project has no measurement of a real approval time. Averaged over all devices the full system halves the manual wait; on high-stakes devices it equals the manual wait, by design.

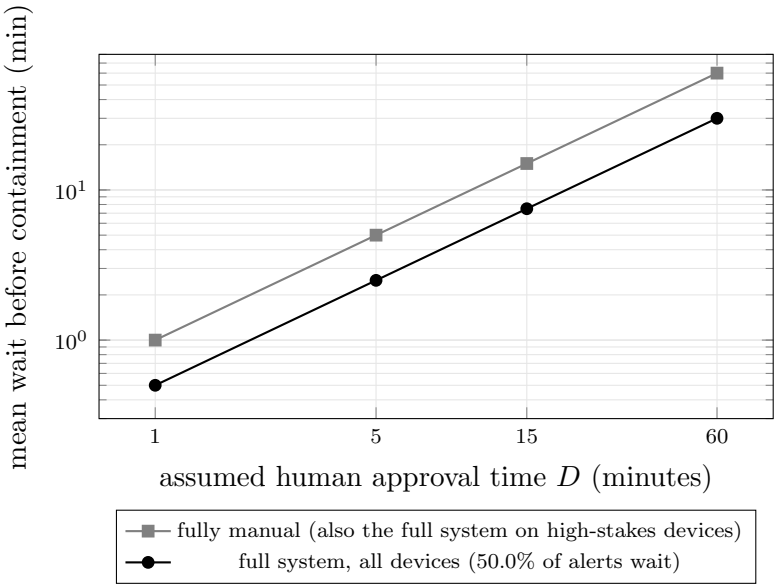


Figure 13: Modelled mean wait before containment against an assumed approval time (fixed playbook: 0, not drawn on the logarithmic axis). The shares of alerts that wait are measured; D is assumed.

5.4. Discussion

Three points follow from the comparisons. First, detection accuracy is no longer the differentiator on CICIoMT2024: the published detectors and ours are all high, and our remaining weaknesses (ARP spoofing, scans) show up only in per-class metrics. Second, the safety result comes from the structure of the system, not from model quality: the Policy Engine's registry-based rule produces the zero unsafe actions regardless of what the agents say (Section 4.9),

and the guards stop wrong agent output from reaching the Policy Engine's inputs. Third, the cost is real and visible: half of alerts wait for a person in the base setting and more in hospitals with many patient-dependent devices (Section 4.8), and false alarms are still isolated at a rate that the calibrated gate only partly reduces. These are the trade-offs a hospital would have to accept or tune, and they are the questions a deployment study with real clinicians must answer.

6. Conclusion

This paper presented a patient-aware framework that turns detection of attacks on medical devices into a safe and auditable response. Its novelty is the evaluated combination of a real detector, untrusted language-model agents corrected by deterministic guards, rule-based engines that place a human before severe actions on patient-dependent devices, and tamper-evident provenance on a real Fabric network. On the full CICIoMT2024 test split the detector reached 99.52% accuracy, and across about 1.3 million high-stakes trials the system took no unsafe autonomous action, against 99.72% for a fixed playbook; an exhaustive check of 3,168 policy inputs confirmed the rule. Under injected faults the guards removed every unsafe action, and every tested ledger tampering was detected. Limits remain: device context and agents were simulated, about half of false alarms are still isolated, and the ledger used one orderer. Future work will evaluate real language models with clinician approval studies on hospital device inventories, and will run the ledger on a multi-orderer network.

References

[1] A. Naghib, F. S. Gharehchopogh, A. Zamanifar, A comprehensive and systematic literature review on intrusion detection systems in the internet of medical things: current status, challenges, and opportunities, Artificial Intelligence Review 58 (2025) 114. doi:10.1007/s10462-024-11101-w.

[2] S. Dadkhah, E. C. P. Neto, R. Ferreira, R. C. Molokwu, S. Sadeghi, A. A. Ghorbani, CICIoMT2024: Attack vectors in healthcare devices – a multi-protocol dataset for assessing IoMT device security, Internet of Things 28 (2024). (Elsevier PII S2542660524002920.)

[3] G. Akar, S. Sahmoud, M. Onat, U. Cavusoglu, E. Malondo, L2D2: A novel LSTM model for multi-class intrusion detection systems in the era of IoMT, IEEE Access 13 (2025) 7002–7013. doi:10.1109/ACCESS.2025.3526883.

[4] J. A. Shaikh, C. Wang, M. W. U. Sima, M. Arshad, M. Owais, D. S. M. Hassan, R. Alkanhel, M. S. A. Muthanna, A deep reinforcement learning-based robust intrusion detection system for securing IoMT healthcare networks, Frontiers in Medicine 12 (2025) 1524286. doi:10.3389/fmed.2025.1524286.

[5] A. Hafid, M. Rahouti, M. Aledhari, Optimizing intrusion detection in IoMT networks through interpretable and cost-aware machine learning, Mathematics 13 (10) (2025) 1574. doi:10.3390/math13101574.

[6] X. Lin, J. Zhang, G. Deng, T. Liu, T. Zhang, Q. Guo, R. Chen, IRCopilot: Automated incident response with large language models, arXiv:2505.20945 (2025), preprint.

[7] Y. Gao, K. Hammar, T. Li, In-context autonomous network incident response: An end-to-end large language model agent approach, arXiv:2602.13156 (2026), preprint.

[8] M. Hasnain, F. R. Albogamy, S. S. Alamri, I. Ghani, B. Mehboob, The Hyperledger Fabric as a blockchain framework preserves the security of electronic health records, Frontiers in Public Health 11 (2023) 1272787. doi:10.3389/fpubh.2023.1272787.

[9] F. Pelekoudas-Oikonomou, J. C. Ribeiro, G. Mantas, G. Sakellari, J. Gonzalez, Prototyping a Hyperledger Fabric-based security architecture for IoMT-based health monitoring systems, Future Internet 15 (9) (2023) 308. doi:10.3390/fi15090308.

[10] S. Vidhya, V. Kalaivani, A blockchain based secure and privacy aware medical data sharing using smart contract and encryption scheme, Peer-to-Peer Networking and Applications (2023). doi:10.1007/s12083-023-01449-1.

[11] A. Mukhopadhyay, R. Vinayaka, A smart-contract-based blockchain for a healthcare IoT network, International Journal of Electronic Healthcare (2021). doi:10.1504/IJEH.2021.117122.

[12] M. M. Pranav, S. Rithan, R. D. A. Raj, A. Pallakonda, R. M. R. Yanamala, K. Prakasha K., Smart grid cybersecurity against power system MiTM threats and machine learning-based attack classification, Energy Reports (2026). doi:10.1016/j.egyr.2025.12.035.

[13] V. M. Vignes, M. P. Sri Harini, R. Satheesh, V. Das, S. Padmanaban, AI-driven cybersecurity framework for anomaly detection in power systems, Scientific Reports (2025). doi:10.1038/s41598-025-19634-y.

[14] T. Alsolami, B. Alsharif, M. Ilyas, Enhancing cybersecurity in healthcare: Evaluating ensemble learning models for intrusion detection in the Internet of Medical Things, Sensors 24 (18) (2024) 5937. doi:10.3390/s24185937.

[15] M. Abdelhaq, S. Palanisamy, M. Gopinath, V. G. S. Manasa, M. A. Ram, S. Khan, et al., A hybrid XGBoost–SVM ensemble framework for robust cyber-attack detection in the internet of medical things (IoMT), Scientific Reports 16 (2026) 6855. doi:10.1038/s41598-026-37832-0.

[16] N. Xiao, Z. Wang, X. Sun, J. Miao, A novel blockchain-based digital forensics framework for preserving evidence and enabling investigation in industrial Internet of Things, Alexandria Engineering Journal 86 (2024) 631–643.